

Emulator Frontend

Project Design Document

Version 1 scope and product boundaries

Technology: C# and Avalonia UI

Primary platform: Windows

Project type: Personal, open-source, portable desktop application

1. Project Overview

This project is a lightweight desktop frontend for organizing and launching emulated games through separately installed emulators. It provides an OpenEmu-inspired library interface, but it does not include emulator cores or perform emulation itself.

The first version is a Windows-only application. The code should avoid unnecessary Windows-specific coupling so Linux and macOS support can be added later without redesigning the entire application.

Core user experience

1. The user adds an individual game or a dedicated system folder.
2. The frontend suggests or confirms the game system.
3. The game appears in the system library.
4. The user double-clicks the game.
5. The frontend launches the configured external emulator with the correct game path.
6. The frontend hides or minimizes while the emulator is running.
7. The frontend restores itself when the emulator exits.

2. Product Goals

- Provide a simple, polished library inspired by OpenEmu.
- Support cover-grid and list views, system filtering, search, and double-click launching.
- Keep system-specific and emulator-specific behavior isolated so new systems can be added later without broad refactoring.
- Remain efficient in memory, CPU, and GPU usage.
- Use portable storage beside the application.
- Leave room for future metadata providers, RetroAchievements, save backups, and cloud synchronization.

3. Supported Systems and Emulators

System	External emulator
PlayStation	DuckStation
PlayStation 2	PCSX2
PlayStation 3	RPCS3
Nintendo GameCube	Dolphin
Nintendo Wii	Dolphin

GameCube and Wii appear as separate systems in the sidebar even though both use Dolphin.

Version 1 supports one configured emulator per system. The design should not prevent multiple emulator choices from being added later.

4. Version 1 Scope

Included

- Windows support.
- Portable-only application storage.
- Light and dark themes.
- OpenEmu-style system sidebar.
- Cover-grid and list views.
- Search.
- Adding individual games and dedicated system folders.
- Recursive folder scanning.
- Manual emulator executable selection.
- Default launch arguments that users can edit.
- Double-click launching.
- Hide or minimize during gameplay and restore after emulator exit.
- Manual title editing and local cover selection.
- System-branded placeholder covers.
- Missing games remain visible and marked unavailable.
- Removing entries from the frontend database without deleting original files.
- Separate entries for regions, revisions, and editions.

Not included

- Built-in emulator cores.
- Metadata scraping or automatic cover downloads.
- Game details pages.
- Favorites or custom collections.
- Controller navigation.
- Play-time tracking or importing.
- RetroAchievements.
- Save backups or cloud synchronization.
- Emulator auto-detection.
- Multiple installations of one emulator.
- Per-game launch arguments.
- Save-state, screenshot, or homebrew management.
- Runtime plugins.
- Automatic updates.
- Linux or macOS releases.

5. User Interface

The main window should closely follow the simplicity of OpenEmu's library view without copying it pixel-for-pixel.

- Left sidebar with supported systems.
- Toolbar with grid/list toggle, search, add-game action, and settings.

- Main content area showing the selected system's games.
- Compact status and error feedback for scans, missing files, and launch failures.

Game interaction

- Single click selects a game.
- Double click launches it.
- Right click opens a compact context menu.
- Title and cover editing use an OpenEmu-style popover or small dialog.
- No separate details screen is needed in version 1.

Game presentation

- Grid view shows cover art and title.
- Games without artwork use a system-branded placeholder containing the game title.
- Unavailable games are clearly marked and cannot be launched.
- List view shows at minimum title, system, and availability.

6. Game Importing

General behavior

- Each system has one or more dedicated folders.
- The user may add an entire folder or an individual game.
- Folder scanning is recursive.
- When adding an individual game, the frontend suggests a system using extension or folder structure, then asks the user to confirm.
- Missing files stay in the library and are marked unavailable.

PlayStation

Support common DuckStation-compatible formats, including:

- .cue
- .chd
- .m3u
- .pbp
- .iso where applicable

When a matching .cue file exists, associated .bin files should not appear as separate games.

PlayStation 2

Support common PCSX2-compatible formats, including:

- .iso
- .chd
- .cso
- .m3u

Multi-disc games

- Version 1 uses .m3u playlists.
- The .m3u file is imported and launched as the game entry.
- Disc files referenced by a recognized playlist are not displayed separately.
- Games without an .m3u may appear as separate disc entries.
- Automatic playlist creation and disc grouping are future features.

PlayStation 3

The frontend must support both RPCS3-managed installed games and standalone games stored in separate folders.

- Scan a folder containing multiple PS3 game directories.
- Add one individual PS3 game directory.
- Recognize valid games using directory structure and available local metadata.
- Treat each recognized game directory as one library entry.

GameCube and Wii

GameCube and Wii use separate system definitions and sidebar entries. Their import rules should remain independent even though both launch through Dolphin.

Removing games

Removing a game only changes the frontend database. The confirmation must explicitly state that the original files will not be deleted. A removed game may reappear after a later folder rescan; ignore-list behavior is postponed.

7. Titles and Covers

- Default titles come from filenames, directory names, or available local metadata.
- Users may edit displayed titles manually.
- Different regions, revisions, and editions remain separate entries.
- Users may assign a local image as a cover.
- Games without covers use a consistent system-branded placeholder with the title.
- Automatic metadata and cover providers are planned for a later version.

8. Emulator Configuration and Launching

- The user manually selects the executable for each emulator.
- The application provides default launch arguments.
- The user may edit those arguments globally.
- The application does not detect emulator installations automatically.
- Per-game custom arguments and multiple emulator installations are deferred.

Supported argument placeholders should include:

{GamePath}

{GameDirectory}

{GameFileName}

{EmulatorDirectory}

Launch flow

8. Check that the game file or directory exists.
9. Check that the emulator is configured.
10. Create the launch command using the emulator integration.
11. Hide or minimize the frontend.
12. Launch the emulator.
13. Wait for the relevant emulator process to exit.
14. Restore the frontend.

The launcher should pass the executable and arguments safely rather than constructing an unrestricted shell command.

9. Internal Modularity

The project does not need a plugin system. Modularity means keeping system-specific and emulator-specific behavior separated from shared application behavior.

Shared application responsibilities

- Games and library records.
- System selection and filtering.
- Folder scanning orchestration.
- Cover handling.
- Settings and portable storage.
- Generic process launching.
- Search and library presentation.

Each system defines

- Stable name and identifier.
- Icon or branding.
- Supported extensions.
- File or folder detection rules.
- Special import behavior where required.
- Compatible emulator integration.

Each emulator integration defines

- Stable name and identifier.
- Supported systems.
- Default launch arguments.
- Executable configuration.
- Launch behavior.
- Process-exit tracking.

Most systems should use a shared scanner. Custom importing code should only be introduced where necessary, such as directory-based PS3 games.

A practical initial project organization is:

```
Frontend.App
Frontend.Core
Frontend.Infrastructure
Frontend.Integrations/
  Systems/PlayStation
  Systems/PlayStation2
  Systems/PlayStation3
  Systems/GameCube
  Systems/Wii
Emulators/DuckStation
Emulators/PCSX2
Emulators/RPCS3
Emulators/Dolphin
```

These may begin as feature folders in one integrations project. They can be split into separate C# projects later if they grow substantially.

10. Storage and Portability

Version 1 is portable-only. User data is stored beside the application in a dedicated data structure.

```
EmulatorFrontend/
  EmulatorFrontend.exe
  Data/library.db
  Covers/
  Cache/
  Logs/
  Settings/
```

Relative paths should be supported where practical so the frontend, emulators, and games can be moved together on the same drive.

The frontend must never modify or delete original game files.

11. Performance Expectations

- Virtualize large grid and list views.
- Load covers asynchronously.
- Cache scaled thumbnails rather than repeatedly decoding full-size artwork.
- Run scans outside the UI thread.
- Debounce search input.
- Avoid unnecessary animations and background polling.
- Do not rescan the complete library on every startup.

- Pause or reduce UI work while the frontend is hidden during gameplay.

12. Future Features

- Linux and macOS support.
- Metadata and cover providers.
- RetroAchievements display and post-session refresh, while emulators remain responsible for actual achievement unlocking.
- Save backups and cloud synchronization.
- Play-time tracking or importing from emulator data where available.
- Automatic .m3u creation and disc grouping.
- Additional systems and emulators.
- Automatic updates.

13. First Development Milestones

15. Create the Avalonia application shell with sidebar, toolbar, grid/list switch, search, and settings.
16. Implement portable settings and library storage.
17. Add system definitions and folder scanning for PlayStation, PS2, GameCube, and Wii.
18. Add PS3 directory-based importing.
19. Implement emulator configuration and external process launching.
20. Add cover placeholders, manual cover selection, title editing, and unavailable-game states.
21. Polish performance, themes, error handling, and portable packaging.

14. Definition of the First Usable Version

Version 1 is complete when a user can run the portable Windows application, configure DuckStation, PCSX2, RPCS3, and Dolphin, add folders or individual games for the five supported systems, browse them in grid or list view, search the library, edit titles and covers, and launch games by double-clicking while the frontend hides or minimizes and restores after emulator exit.